Double-Cut-And-Join 操作による ゲノム間距離に関する 線形時間アルゴリズムの再実装

2025 年度 卒業論文

平田 智也

慶應義塾大学 理工学部 数理科学科
小田研究室

2026 年 1 月 27 日

研究背景と主な成果

2 つのゲノム間の近さ（距離）に関する研究について

研究の経緯

- Oliveira らのサーベイ論文（2024）を調査
- 特に研究成果が豊富な **DCJ (Double Cut and Join)** に着目
- Bergeron ら (2009) の線形時間アルゴリズムを参照

本研究の動機

- 既存のアルゴリズム（Bergeron ら (2009)）は記述が**過度に複雑**であり，**より簡素で分かりやすいアルゴリズム**を自分で実装することを目指した.

主な成果

- DCJ 距離および操作手順を，遺伝子数 N に対して線形時間 $O(N)$ で出力するアルゴリズムを，C 言語で概ね独力により実装した.

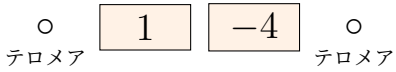
ゲノムの表現方法

ゲノムと染色体

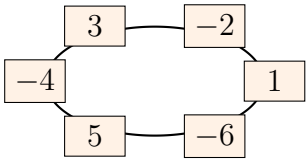
- ゲノムは**染色体**の集合であり，染色体は**遺伝子**の並び.
- 遺伝子は正負の整数で識別（*target* ゲノムにおいて昇順）.
- 線形染色体 $\langle \dots \rangle$ と環状染色体 $(\dots)$ の二つに区別して考える.

$N = 5$ の場合のゲノム表現の例:

- $A : \langle 1\ -4 \rangle\ (3\ -2)\ \langle 5 \rangle$
- $B : \langle 1\ 2\ 3 \rangle\ (4\ 5)$



線形染色体 $\langle 1\ -4 \rangle$



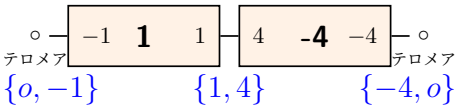
環状染色体 $(1\ -2\ 3\ -4\ 5\ -6)$

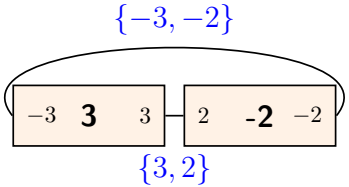
端点と隣接

端点 (extremity) と 隣接 (adjacency)

- **端点**: 遺伝子 g の右端 (g) と左端 $(-g)$.
- **隣接**: 接続された 2 つの端点の集合 $\{u, v\}$.
- 線形染色体の末端は**テロメア**[a] o との隣接 $\{o, x\}$ で表現.

[a]生物学において染色体の末端を保護する構造体に由来する名称





ゲノムの隣接集合による表現

$N = 5$ の場合のゲノム表現の例:

- $A : \langle 1\ \ -4 \rangle\ (3\ \ -2)\ \langle 5 \rangle$
- $B : \langle 1\ 2\ 3 \rangle\ (4\ 5)$

隣接集合による表現

- $A = \{\{o, -1\}, \{1, 4\}, \{-4, o\}, \{3, 2\}, \{-2, -3\}, \{o, -5\}, \{5, o\}\}$
- $B = \{\{o, -1\}, \{1, -2\}, \{2, -3\}, \{3, o\}, \{4, -5\}, \{5, -4\}\}$

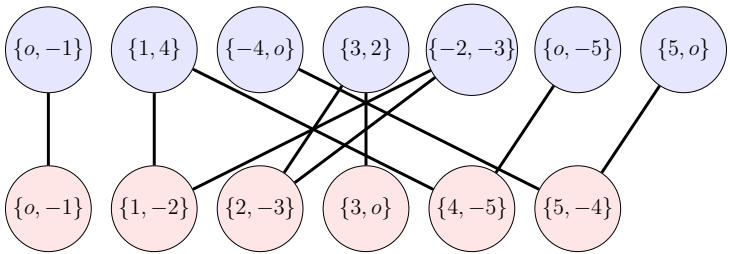
ゲノムを隣接の集合として表す利点

隣接集合による表現を用いることにより，**染色体の種類，及び本数に関係なく**，統一的にゲノムを扱うことができるようになる．また，DCJ 操作も隣接に作用する操作として扱うことができるようになる．

隣接グラフと例

Definition 1 (隣接グラフ $AG(A, B)$)

2 つのゲノム A, B のすべての隣接を頂点集合とし，共通する端点を持つ隣接同士の間にエッジを引いたグラフを隣接グラフ $AG(A, B)$ とする．



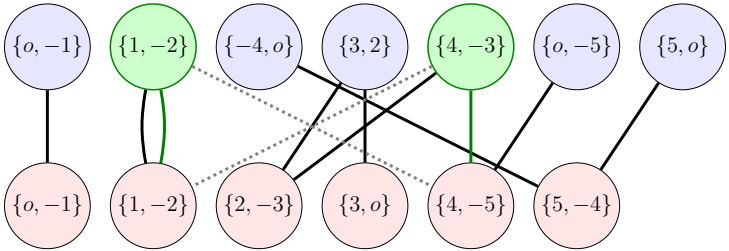
隣接グラフ $AG(A, B)$ は，ゲノム A の隣接集合とゲノム B の隣接集合からなる二部グラフとなる．また，各頂点の次数は最大 2 であるため，隣接グラフは**パスまたはサイクルのみから構成される．**

DCJ 操作の定義と適用例

Definition 2 (Double-Cut-And-Join)

2 つの隣接 $\{u, v\}$ と $\{x, y\}$ を切断し，新たに $\{u, x\}$ と $\{v, y\}$，または $\{u, y\}$ と $\{v, x\}$ として再結合する操作.

隣接 $\{1, 4\}$ と $\{-2, -3\}$ に対して DCJ 操作を適用し，$\{1, -2\}$ と $\{4, -3\}$ に組み替える.



DCJ 距離の定義と本研究の課題

Definition 3 (DCJ 距離 d_{DCJ})

2 つのゲノム A, B 間で，A に DCJ 操作を繰り返し適用することにより B に変換するために必要な最小操作回数を $d_{\mathrm{DCJ}}(A, B)$ とする．

本研究の主題は，任意の 2 つのゲノム A, B に対して，この $d_{\mathrm{DCJ}}(A, B)$ を効率的に計算・出力するアルゴリズムを設計・実装することである．

DCJ距離公式とそれに基づく実装アプローチ1

Theorem 4 (DCJ 距離公式 (Bergeron, 2006))

2つのゲノム A, B 間の最短の DCJ 操作回数 $d_{\mathrm{DCJ}}(A, B)$ は，次式で与えられる：

$$d_{\mathrm{DCJ}}(A, B) = N - \left(C + \frac{I}{2} \right)$$

ここで N は遺伝子数，C は隣接グラフ上のサイクル数，I は奇数長パスの数である．

アプローチ1（距離公式に基づく実装）

初期状態の隣接グラフ $AG(A, B)$ を構築し，その成分（C と I）を数え上げれば，操作シミュレーションを行わずとも距離 d は求まる．

- 具体的な操作手順は出力されないが，グラフ走査のみで完結する．
- **まずはこの手法を考案し，実装を行なった．**

Bergeron らに基づく実装アプローチ 2

アプローチ 2（Bergeron ら (2006) に基づく貪欲法）

距離を 1 減少させる（$C + I/2$ を 1 増加させる）DCJ 操作を Greedy に選択していき，グラフを動的に書き換えていく．その過程で操作手順も出力することができる．

- 隣接の繋ぎ変えを追跡することで，具体的な変形操作の系列を出力可能．
- **距離計算の実装に続き，こちらも実装を行なった．**

計算量について

いずれの手法についても，入力サイズ N に対して線形時間 $O(N)$ で動作するアルゴリズムとして実装できた．

DCJ距離アルゴリズム実装と課題

上述のアプローチ 1，2 を C 言語により実装した．実行時間のオーダーはどちらも線形時間 $O(N)$ である．ゲノム間の隣接グラフなどを走査し，アプローチ 1 は距離のみ，アプローチ 2 は距離と操作手順を出力する．
詳細なソースコードは以下の GitHub リポジトリで公開している：

- `https://github.com/hiratannnnnn/dcj_bergeron.git`

課題

- 現状の実装ではテキスト入力のパースを用いているため，遺伝子番号の桁数に起因して，入力時に見かけ上の対数的なコストが発生している．固定長整数の形式にするなど，桁数に依存しないようにすれば，厳密な線形時間化が可能になる．

参考文献 I

- A. Bergeron, J. Mixtacki, and J. Stoye, “A unifying view of genome rearrangements,” in: *Proceedings of WABI 2006*, LNBI, vol. 4175, Springer Verlag, 2006, pp. 163–173.
- A. Bergeron, J. Mixtacki, and J. Stoye, “A new linear time algorithm to compute the genomic distance via the double cut and join distance,” *Theoretical Computer Science*, 410 (2009), pp. 5300–5316.
- S. Hannenhalli and P. A. Pevzner, “Transforming men into mice (polynomial algorithm for genomic distance problem),” in: *Proceedings of FOCS 1995*, IEEE Press, 1995, pp. 581–592.
- A. R. Oliveira, K. L. Brito, A. O. Alexandrino, G. Siqueira, U. Dias, and Z. Dias, “Rearrangement Distance Problems: An Updated Survey,” *CSUR*, 2024.

DCJ 距離の下限

Lemma 5 (DCJ 距離の下限)

任意の 2 つのゲノム A, B について，DCJ 距離は以下を満たす：

$$d_{\mathrm{DCJ}}(A, B) \geq N - \left(C + \frac{I}{2}\right)$$

ここで，N は遺伝子数，C は**隣接グラフ** $AG(A, B)$ **上のサイクル**の個数，I は**隣接グラフ** $AG(A, B)$ **上の奇数長パス**の個数である．

成分の変化量に関する性質

- DCJ 操作によってサイクル数 C と奇数長パス数 I が**同時に増減することはない**．
- サイクル数 C が増減するときは，最大で 1 しか変わらない ($\Delta C \in \{0, \pm 1\}$)．
- 奇数長パス数 I が変化するときは，$2, 0, -2$ しか変わらない ($\Delta I \in \{0, \pm 2\}$)．

アルゴリズムの正当性の根拠

整数性

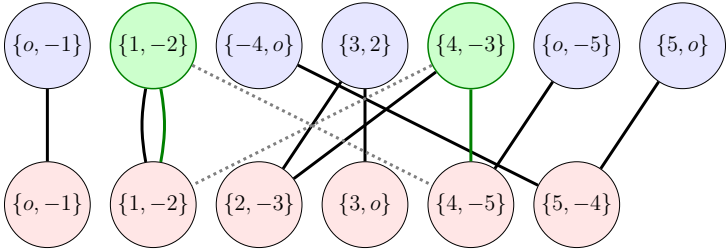
$C + I/2$ の変化量は常に 0 または ± 1 であり，DCJ 操作ごとに距離 d も整数として保たれる．

最適性の保証

任意の $A \neq B$ に対し，必ず d を 1 減少させる DCJ 操作が存在する．したがって，距離公式の下限値は実際に達成可能である．

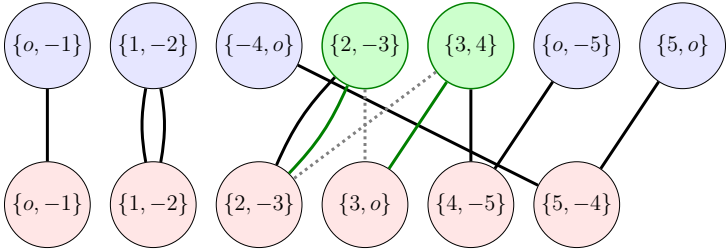
したがって，Lemma 3 と以上の議論より，次の定理が導かれる．

DCJ 操作手順 1



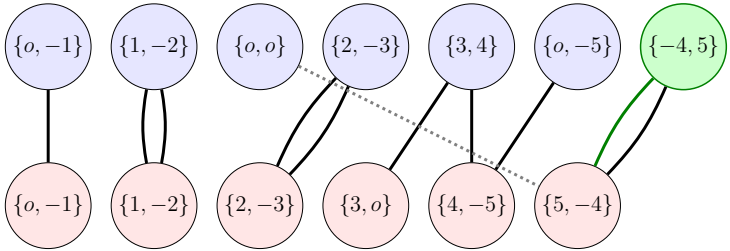
2 回目の DCJ 操作適用後：$(1,4),\ (-2,-3)\ \rightarrow\ (1,-2),\ (4,-3)$

DCJ 操作手順 2



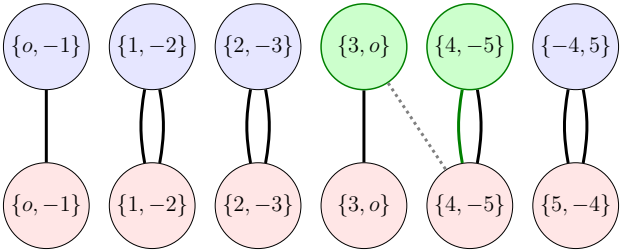
2 回目の DCJ 操作適用後：$(2,3),\ (-3,4)\ \rightarrow\ (2,-3),\ (3,o)$

DCJ 操作手順 3



3 回目の DCJ 操作適用後：$(-4, o),\ (5, o)\ \rightarrow\ (-4, 5),\ (o, o)$

DCJ 操作手順 4



4 回目の DCJ 操作適用後：$(4, 3),\ (-5, o) \ \rightarrow\ (4, -5),\ (3, o)$

benchmark-読み込み含む

Table: 全体実行時間（読み込み・構築含む）とメモリ使用量

遺伝子数 (N)	実行時間 (sec)	メモリ使用量 (MB)
10	0.001	0.003
100	0.001	0.03
1,000	0.007	0.26
10,000	0.179	2.55
100,000	20.827	25.55

benchmark-読み込み除く

Table: アルゴリズム実行時間（読み込み・構築除く）

遺伝子数 (N)	実行時間 (sec)
10	0.001
100	0.001
1,000	0.002
10,000	0.005
100,000	0.051